

e-Hotels Course Project: Deliverable 2 Report

Technologies Used

For this project, I used PostgreSQL for the database management system. The backend logic and server communication were written in Java, and the frontend user interface was built using standard HTML. To host the web application and handle the client-server requests, I used an Apache Tomcat server.

Installation and Setup Steps

Getting this project to run involves setting up the database and then launching the web server. Here is how to do it:

1. Open pgAdmin and create a new database called ehotels.
2. Open the Query Tool and run the SQL scripts provided below to build the tables, triggers, and insert the initial data.
3. Open Eclipse IDE and import the provided project folder.
4. Set up an Apache Tomcat v10.1 server in Eclipse. Make sure the server is pointed to the correct inner bin/lib folder, or it will throw a Bootstrap error.
5. Because I used Tomcat 10.1, I ran into a major snag during development. The default Java Servlet code uses "javax.servlet" imports, which crashed my server. I figured out that Tomcat 10 requires the newer Jakarta EE, so I had to go into all of my Java files and manually change the imports from "javax.servlet" to "jakarta.servlet". If you are running this, ensure your Tomcat version matches this requirement!
6. Right-click the index.html file in the project explorer and select "Run on Server" to launch the application.

The Development Journey

Building the application was an iterative process, especially for the Employee portal. I didn't just write it all at once. First, I designed the basic HTML dashboard just to handle customer check-ins. Once I got the HTML looking right, I built the CheckInServlet in Java to handle the logic.

After testing the check-in process and confirming my database triggers were successfully stopping overlapping rentings, I went back to the drawing board. I reopened the employee HTML file and added a new section for processing payments, and then created a separate Java

servlet just for that. Finally, I added the buttons to view the Management Reports. Taking it step-by-step like this helped me isolate bugs (like the import version issue mentioned above) much faster than if I had tried to code the entire portal in one sitting.

Database Implementation (SQL Code)

Below is the SQL code that supports the functionalities of my application, including the constraints, population, queries, views, and indexes.

1. Database Triggers (User-Defined Constraints)

Preventing Overlapping Bookings: This ensures no two customers can book the same room for overlapping dates.

```
CREATE OR REPLACE FUNCTION check_booking_overlap()

RETURNS TRIGGER AS $$

BEGIN

IF EXISTS (

    SELECT 1 FROM Booking

    WHERE room_id = NEW.room_id

    AND booking_id != COALESCE(NEW.booking_id, -1)

    AND (NEW.start_date < end_date AND NEW.end_date > start_date)

) THEN

    RAISE EXCEPTION 'Constraint Violation: Room % is already booked during these dates.',
NEW.room_id;

END IF;

RETURN NEW;

END;

$$ LANGUAGE plpgsql;

CREATE TRIGGER prevent_booking_overlap
```

BEFORE INSERT OR UPDATE ON Booking

FOR EACH ROW EXECUTE FUNCTION check_booking_overlap();

Enforcing Manager Hierarchy: This ensures an employee and their manager are actually assigned to the same hotel.

CREATE OR REPLACE FUNCTION check_manager_hotel()

RETURNS TRIGGER AS \$\$

DECLARE

mgr_hotel_id INT;

BEGIN

IF NEW.manager_id IS NOT NULL THEN

SELECT hotel_id INTO mgr_hotel_id

FROM Employee

WHERE employee_id = NEW.manager_id;

IF mgr_hotel_id != NEW.hotel_id THEN

RAISE EXCEPTION 'Constraint Violation: An employee and their manager must belong to the same hotel.';

END IF;

END IF;

RETURN NEW;

END;

\$\$ LANGUAGE plpgsql;

CREATE TRIGGER enforce_manager_hotel

BEFORE INSERT OR UPDATE ON Employee

FOR EACH ROW EXECUTE FUNCTION check_manager_hotel();

Preventing Overlapping Rentings: This stops a walk-in renting or check-in from overlapping with an existing active renting.

SQL

CREATE OR REPLACE FUNCTION check_renting_overlap()

RETURNS TRIGGER AS \$\$

BEGIN

IF EXISTS (

 SELECT 1 FROM Renting

 WHERE room_id = NEW.room_id

 AND renting_id != COALESCE(NEW.renting_id, -1)

 AND (NEW.start_date < end_date AND NEW.end_date > start_date)

) THEN

 RAISE EXCEPTION 'Constraint Violation: Room % is currently being rented during these dates.', NEW.room_id;

END IF;

RETURN NEW;

END;

\$\$ LANGUAGE plpgsql;

CREATE TRIGGER prevent_renting_overlap

BEFORE INSERT OR UPDATE ON Renting

FOR EACH ROW EXECUTE FUNCTION check_renting_overlap();

2. Database Population Snippets

I populated 5 hotel chains and generated 8 hotels per chain for a total of 40 hotels. The math in my query ensured they were split across 3 categories (stars), and multiple hotels were placed in the same area like Downtown Ottawa. I also generated 5 rooms per hotel, ensuring they had different capacities ranging from 1 to 5.

Here is an example of the query I used to generate the 200 rooms:

```
INSERT INTO Room (hotel_id, price, capacity, view_type, amenities, is_extendable,
problems_status)

SELECT

    h.hotel_id,

    (r.cap * 50.00) + 100.00 AS price,

    r.cap AS capacity,

    CASE WHEN r.cap % 2 = 0 THEN 'Sea View' ELSE 'Mountain View' END AS view_type,

    'TV, AC, Mini-fridge, Wi-Fi',

    CASE WHEN r.cap > 2 THEN true ELSE false END AS is_extendable,

    'None'

FROM Hotel h

CROSS JOIN generate_series(1, 5) AS r(cap);
```

3. Database Queries

Query 1: Aggregation with GROUP BY Finds the total rooms and average price grouped by hotel star ratings.

```
SELECT

    h.stars AS hotel_category_stars,

    COUNT(r.room_id) AS total_rooms,

    ROUND(AVG(r.price), 2) AS average_room_price

FROM Hotel h

JOIN Room r ON h.hotel_id = r.hotel_id

GROUP BY h.stars

ORDER BY h.stars DESC;
```

Query 2: Nested Query Finds the names and addresses of customers who have booked a room that costs more than \$200.

```
SELECT full_name, address

FROM Customer

WHERE customer_id IN (

    SELECT b.customer_id

    FROM Booking b

    JOIN Room r ON b.room_id = r.room_id

    WHERE r.price > 200.00

);
```

Query 3: Multi-Table JOIN Pulls a detailed report showing the customer name, hotel address, room details, and the employee who handled the renting.

```
SELECT
```

```
c.full_name AS customer_name,  
h.address AS hotel_address,  
r.room_id,  
r.view_type,  
e.full_name AS handled_by_employee,  
rent.start_date,  
rent.end_date  
FROM Renting rent  
JOIN Customer c ON rent.customer_id = c.customer_id  
JOIN Room r ON rent.room_id = r.room_id  
JOIN Hotel h ON r.hotel_id = h.hotel_id  
JOIN Employee e ON rent.employee_id = e.employee_id;
```

Query 4: Aggregation with HAVING Finds hotel chains that have a total combined capacity of more than 30 rooms across all their locations.

```
SELECT  
    hc.chain_id,  
    hc.office_address,  
    SUM(h.num_rooms) AS total_rooms_in_chain  
FROM HotelChain hc  
JOIN Hotel h ON hc.chain_id = h.chain_id  
GROUP BY hc.chain_id, hc.office_address  
HAVING SUM(h.num_rooms) > 30;
```

4. Database Views

View 1: Available Rooms Per Area This view calculates the number of rooms currently available today by excluding any rooms that have an active booking or renting for the current date.

```
CREATE OR REPLACE VIEW Available_Rooms_Per_Area AS

SELECT

    h.address AS area,

    COUNT(r.room_id) AS available_rooms

FROM Hotel h

JOIN Room r ON h.hotel_id = r.hotel_id

WHERE r.room_id NOT IN (

    SELECT room_id FROM Booking WHERE CURRENT_DATE >= start_date AND
    CURRENT_DATE < end_date

    UNION

    SELECT room_id FROM Renting WHERE CURRENT_DATE >= start_date AND
    CURRENT_DATE < end_date

)

GROUP BY h.address;
```

View 2: Aggregated Capacity by Hotel This view calculates the total guest capacity for each specific hotel location.

```
CREATE OR REPLACE VIEW Hotel_Aggregated_Capacity AS

SELECT

    h.hotel_id,

    h.address AS hotel_address,

    SUM(r.capacity) AS total_capacity

FROM Hotel h
```

```
JOIN Room r ON h.hotel_id = r.hotel_id
```

```
GROUP BY h.hotel_id, h.address;
```

5. Database Indexes and Justifications

Expected Queries & Updates: In an online hotel booking system, the most frequent operations are SELECT queries where customers search for available rooms. Updates happen when someone checks in, but the database is mostly doing read operations. I chose indexes that speed up the most common search filters.

Index 1: Hotel Address

```
CREATE INDEX idx_hotel_address ON Hotel(address);
```

Justification: Customers constantly filter searches by city or area. Without this, the database does a slow sequential scan. This index lets the database instantly locate hotels by area.

Index 2: Room Price

```
CREATE INDEX idx_room_price ON Room(price);
```

Justification: Price is a primary filter when users are trying to stay within a budget or sorting from lowest to highest. This index prevents full table scans and gets results back in a fraction of the time.

Index 3: Booking Dates

```
CREATE INDEX idx_booking_dates ON Booking(start_date, end_date);
```

Justification: The core feature of the app is checking if a room is available on specific dates. A composite index on the start and end dates makes overlap-checking highly efficient when the system validates new bookings.

6. Final UI Polish (CSS and JavaScript)

After I finished all the backend Java and SQL stuff and got the whole project working, I realized the raw HTML pages looked pretty ugly. It honestly looked a bit like a plain text resume. I wanted it to look like a real, usable web app before submitting it, so I went back in and added some quick frontend polish.

Adding a Safety Net (JavaScript)

Because my database has cascading deletes set up, clicking a "Delete Hotel" button by accident would be a disaster—it would wipe out all the rooms and employees attached to it. To fix this, I wrote a quick JavaScript snippet that checks every form on the page. If it sees that a form is trying to "delete" a record, it intercepts the click and throws a browser confirmation pop-up asking if you are absolutely sure. If you hit cancel, the script stops the submission dead in its tracks.

Making it Look Modern (CSS)

I kept the styling simple so I wouldn't have to deal with massive frameworks. I set up some CSS root variables at the top to stick to a consistent color theme (a dark blue primary color with orange accent buttons).

Instead of having a giant, overwhelming wall of text on the Admin page, I wrapped the forms inside "cards" with a white background and a soft shadow to separate them. I also styled the input boxes to have a nice blue outline when you click into them, and I formatted the data tables with a solid colored header and row hover effects so the SQL query results are actually easy to read.